

Fluxo Completo da Arquitetura - TrocaAi

FRONTEND (Vue.js + TypeScript)



▶ ****index.html**** - Ponto de Entrada da Aplicação
(Carrega o JavaScript principal via `<script src="/src/main.ts">`)

▶ ****main.ts**** - Inicializador do Vue.js

- Cria a aplicação Vue: `createApp(App)`
- Instala o Pinia: `app.use(createPinia())` (Gerenciamento de Estado Global)
- Instala o Vue Router: `app.use(router)` (Navegação entre páginas)
- Conecta chat ao montar: `chatStore.connect()` (WebSocket)
- Monta a aplicação: `app.mount('#app')`

▶ ****App.vue**** - Layout Principal Global

- Renderiza componentes globais fixos:

▶ ****AppHeader.vue**** (Cabeçalho)

- Logo e navegação
- Botão "Novo Item"
- Badge de notificações (`notificationStore.unreadCount + chatStore.unreadCount`)
- Dropdown do usuário (Perfil, Logout)
- Menu Admin (se `authStore.isAdmin`)

▶ Depende de: `'stores/auth.ts'`, `'stores/notification.ts'`, `'stores/chat.ts'`

▶ ****AppFooter.vue**** (Rodapé)

- Links rápidos
- Informações do projeto

▶ Copyright 2025

▶ ****FloatingChat.vue**** (Chat Flutuante)

- Badge de mensagens não lidas (`chatStore.unreadCount`)
- Lista de conversas (`chatStore.conversations`)
- Janela de mensagens (`chatStore.currentMessages`)
- Input de envio (`chatStore.sendMessage`)
- Status online/offline (`chatStore.onlineUsers`)
- Indicador "digitando..." (`chatStore.typingUsers`)

▶ Depende de: `'stores/chat.ts'`, `'stores/auth.ts'`

▶ ****<router-view />**** - Ponto de Injeção das Views

(O Vue Router injeta dinamicamente o componente da rota atual aqui)

▶ ****router/index.ts**** (Vue Router) - O Cérebro da Navegação

- Mapeia URLs para componentes de Visão (.vue)
- Exemplos de rotas:
 - `'/'` → `HomeView.vue`
 - `'/login'` → `LoginView.vue`
 - `'/items/:id'` → `ItemDetailsView.vue`

- `/meus-itens` → MyItemsView.vue (protegida)
- `/admin` → AdminView.vue (protegida + admin)

▶ ****Guard: beforeEach**** - Proteção de Rotas

- Verifica `authStore.isAuthenticated` antes de acessar rotas protegidas
- Verifica `authStore.isAdmin` para rotas administrativas
- Redireciona para `/login` se não autenticado
- ****Depende de:**** `stores/auth.ts`

▶ ****views/*.vue**** - As Páginas da Aplicação

▶ ****PÁGINAS PÚBLICAS**** (Não requerem autenticação)

▶ ****HomeView.vue**** - Página Inicial

- Lista todos os itens disponíveis
- Busca por título
- Filtros por categoria
- Paginação

▶ Chama: `itemStore.fetchItems()`

▶ ****MapView.vue**** - Mapa de Itens

- Exibe itens no mapa (lat/lng)
- Raio de busca ajustável
- Cálculo de distância
- Filtros por categoria

▶ Chama: `itemStore.fetchItems()`, usa `ItemsMap.vue`

▶ ****ItemDetailsView.vue**** - Detalhes do Item

- Exibe informações completas do item
- Carrossel de imagens
- Botão "Fazer Proposta" (se autenticado)
- Perfil do dono do item

▶ Chama: `itemStore.fetchItemById()`, `proposalStore.createProposal()`

▶ ****LoginView.vue**** - Login

- Formulário de login (email + senha)
- Validação de campos

▶ Chama: `authStore.login()`

▶ ****RegisterView.vue**** - Cadastro

- Formulário de cadastro (nome, email, senha)
- Validação de campos

▶ Chama: `authStore.register()`

▶ ****PÁGINAS AUTENTICADAS**** (Requerem autenticação)

****MyItemsView.vue**** - Meus Itens

- Lista itens do usuário logado
- Botões: Editar, Deletar, Ver
- Badge de status (disponível/indisponível)

▶ Chama: ``itemStore.fetchMyItems()`, `itemStore.deleteItem()``

****NewItemView.vue**** - Cadastrar Item

- Formulário completo de cadastro
- Upload múltiplo de imagens (Multer no backend)
- Seleção de categoria
- Geolocalização (`navigator.geolocation`)

▶ Chama: ``itemStore.createItem(formData)``

****EditItemView.vue**** - Editar Item

- Carrega dados do item
- Permite edição de campos

▶ Chama: ``itemStore.fetchItemById()`, `itemStore.updateItem()``

****MyProposalsView.vue**** - Minhas Propostas Enviadas

- Lista propostas que o usuário fez
- Status: pendente, aceita, recusada
- Botão para abrir chat

▶ Chama: ``proposalStore.fetchSentProposals()``

****ReceivedProposalsView.vue**** - Propostas Recebidas

- Lista propostas nos itens do usuário
- Botões: Aceitar, Recusar (se pendente)
- Modal com detalhes completos
- Abre chat automaticamente ao aceitar

▶ Chama: ``proposalStore.fetchReceivedProposals()`,
`proposalStore.respondToProposal()`,
`chatStore.openChatWithConversation()``

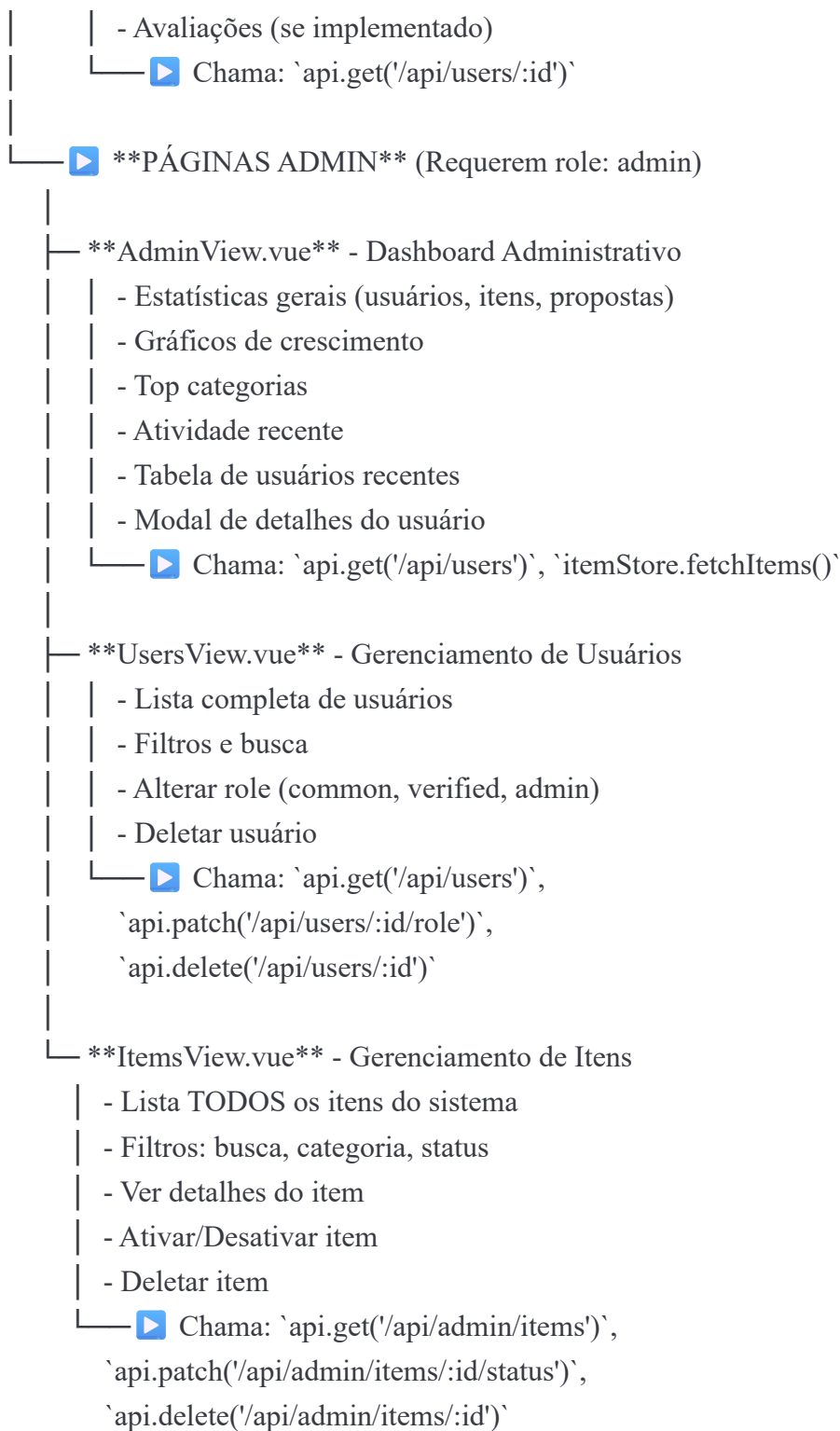
****ProfileView.vue**** - Perfil do Usuário

- Dados do usuário logado
- Edição de informações
- Estatísticas (itens, propostas)

▶ Chama: ``authStore.user`, `authStore.updateUser()``

****UserProfileView.vue**** - Perfil Público de Outro Usuário

- Visualização de perfil público
- Itens do usuário



STORES (Pinia) - Gerenciamento de Estado



 ****auth.ts**** - Autenticação e Usuário

****Estado:****

- user: User | null (dados do usuário logado)
- token: string | null (JWT)
- isAuthenticated: boolean
- isAdmin: boolean

****Ações:****

- login(email, senha) → POST /api/auth/login
- register(nome, email, senha) → POST /api/auth/register
- logout() → limpa localStorage e redireciona
- checkAuth() → GET /api/auth/me (verifica token)

 Depende de: `services/api.ts`, `localStorage`

 ****item.ts**** - Gerenciamento de Itens

****Estado:****

- items: Item[] (lista de itens)
- myItems: Item[] (itens do usuário)
- currentItem: Item | null (item sendo visualizado)
- loading: boolean
- pagination: { page, limit, total, totalPages }

****Ações:****

- fetchItems(filters) → GET /api/items?page=1&category=X
- fetchItemById(id) → GET /api/items/:id
- fetchMyItems() → GET /api/items/my
- createItem(formData) → POST /api/items (multipart/form-data)
- updateItem(id, data) → PUT /api/items/:id
- deleteItem(id) → DELETE /api/items/:id
- fetchCategories() → GET /api/items/categories

 Depende de: `services/api.ts`

 ****proposal.ts**** - Propostas de Troca

****Estado:****

- sentProposals: Proposal[] (propostas enviadas)
- receivedProposals: Proposal[] (propostas recebidas)
- loading: boolean

****Ações:****

- fetchSentProposals() → GET /api/proposals/sent

- fetchReceivedProposals() → GET /api/proposals/received
- createProposal(data) → POST /api/proposals
- respondToProposal(id, status) → PATCH /api/proposals/:id/respond (status: 'aceita' | 'recusada')

Depende de: `services/api.ts`, `stores/notification.ts`

▶ ****chat.ts**** - Chat em Tempo Real

****Estado:****

- socket: Socket | null (conexão WebSocket)
- conversations: Conversation[] (lista de conversas)
- currentMessages: ChatMessage[] (mensagens da conversa atual)
- onlineUsers: Set<number> (IDs de usuários online)
- typingUsers: Map<string, boolean> (quem está digitando)
- isChatOpen: boolean
- selectedConversation: Conversation | null
- conversationToOpen: { otherUserId, itemId } | null

****Computed:****

- connected: boolean (socket?.connected)
- unreadCount: number (soma de não lidas)

****Ações:****

- connect() → Conecta ao WebSocket (Socket.IO)
- disconnect() → Desconecta
- fetchConversations() → GET /api/chat/conversations
- fetchMessages(otherUserId, itemId) → GET /api/chat/messages/:userId/:itemId
- sendMessage(otherUserId, itemId, content) → Emite evento 'message:send'
- markAsRead(otherUserId, itemId) → POST /api/chat/read + emit 'message:read'
- notifyTyping(otherUserId, itemId, isTyping) → emit 'user:typing'
- openChatWithConversation(otherUserId, itemId) → Sinaliza para abrir chat

****Listeners WebSocket:****

- 'connect' → Solicita lista de online
- 'disconnect' → Log
- 'message:received' → Adiciona mensagem, atualiza conversas
- 'unread:update' → Atualiza contador
- 'user:online_list' → Atualiza Set de online
- 'user:online' → Adiciona ao Set
- 'user:offline' → Remove do Set
- 'user:typing' → Atualiza Map de digitando

Depende de: `services/api.ts`, `socket.io-client`, `stores/auth.ts`

▶ ****notification.ts**** - Notificações

****Estado:****

- notifications: Notification[] (lista de notificações)
- loading: boolean

****Computed:****

- unreadCount: number (notificações não lidas)

****Ações:****

- fetchNotifications() → GET /api/notifications
- markAsRead(id) → PATCH /api/notifications/:id/read

▶ Depende de: `services/api.ts`



SERVICES - Camada de Comunicação



📁 ****services/****

▶ ****api.ts**** - O Mensageiro para o Backend

****Configuração:****

- Cria instância do Axios
- baseURL: ``import.meta.env.VITE_API_URL || 'http://localhost:3000'``
- timeout: 10000ms
- headers: `{ 'Content-Type': 'application/json' }`

****Interceptors de Request:****

- Antes de cada requisição:
 1. Busca token do localStorage
 2. Se token existe, adiciona header: ``Authorization: Bearer ${token}``
 3. Log da requisição (em dev)

****Interceptors de Response:****

- Em caso de erro 401 (Não autorizado):
 1. Remove token do localStorage
 2. Redireciona para /login
- Em caso de outros erros:
 1. Log do erro
 2. Repassa o erro para o caller

🔗 ENVIA REQUISIÇÃO HTTP 🔗



⚙️ **BACKEND (Node.js + Express + TypeORM)**





****HTTP REQUEST CHEGA NO BACKEND****



****backend/src/server.ts**** - Servidor Principal

- Inicializa Express
- Configura CORS
- Configura Middlewares (body-parser, express.json)
- Cria servidor HTTP: ``createServer(app)``
- Inicializa Socket.IO: ``new Server(httpServer)``
- Conecta ao banco: ``AppDataSource.initialize()``
- Registra rotas: ``app.use('/api', routes)``
- Inicializa WebSocket: ``new ChatSocketHandler(io)``
- Escuta na porta 3000



****backend/src/routes/index.ts**** - Roteador Principal

- Importa todos os controllers
- Mapeia rotas para controllers



****ROTAS DE AUTENTICAÇÃO**** (/api/auth/*)

- └ POST /api/auth/register → userController.register()
- └ POST /api/auth/login → userController.login()
- └ GET /api/auth/me → authMiddleware → userController.getMe()



****ROTAS DE ITENS**** (/api/items/*)

- └ GET /api/items → itemController.findAll()
(Aceita query params: page, limit, category, search, status)
- └ GET /api/items/:id → itemController.findOne()
- └ POST /api/items → authMiddleware → itemController.create()
(Upload de imagens via Multer middleware)
- └ PUT /api/items/:id → authMiddleware → itemController.update()
- └ DELETE /api/items/:id → authMiddleware → itemController.delete()
- └ GET /api/items/my → authMiddleware → itemController.findMyItems()
- └ PATCH /api/items/:id/status → authMiddleware → itemController.updateStatus()



****ROTAS DE PROPOSTAS**** (/api/proposals/*)

- └ GET /api/proposals/sent → authMiddleware → proposalController.findSent()
- └ GET /api/proposals/received → authMiddleware → proposalController.findReceived()
- └ POST /api/proposals → authMiddleware → proposalController.create()
- └ PATCH /api/proposals/:id/respond → authMiddleware → proposalController.respond()
(body: { status: 'aceita' | 'recusada' })



****ROTAS DE CHAT**** (/api/chat/*)

- GET /api/chat/conversations → authMiddleware → chatController.getConversations()
- GET /api/chat/messages/:otherUserId/:itemId → authMiddleware → chatController.getMessages()
- POST /api/chat/read → authMiddleware → chatController.markAsRead()
- GET /api/chat/unread-count → authMiddleware → chatController.getUnreadCount()
- DELETE /api/chat/conversation/:otherUserId/:itemId → authMiddleware → chatController.deleteConversation()

▶ ****ROTAS DE NOTIFICAÇÕES**** (/api/notifications/*)

- GET /api/notifications → authMiddleware → notificationController.findByUser()
- PATCH /api/notifications/:id/read → authMiddleware → notificationController.markAsRead()

▶ ****ROTAS ADMIN**** (/api/admin/*)

- GET /api/admin/items → authMiddleware → requireAdmin → adminController.getAllItems()
- DELETE /api/admin/items/:id → authMiddleware → requireAdmin → adminController.deleteItem()
- GET /api/users → authMiddleware → requireAdmin → userController.findAll()
- PATCH /api/users/:id/role → authMiddleware → requireAdmin → userController.updateRole()
- DELETE /api/users/:id → authMiddleware → requireAdmin → userController.delete()

▶ ****MIDDLEWARE DE AUTENTICAÇÃO****

▶ ****authMiddleware**** (backend/src/middlewares/auth.middleware.ts)

1. Extrai token do header Authorization
2. Verifica token com JWT
3. Decodifica payload: { userId, role }
4. Busca usuário no banco
5. Anexa userId e role ao request
6. Chama next() ou retorna 401

▶ Se autenticado → Controllers

▶ ****requireAdmin**** (backend/src/middlewares/auth.middleware.ts)

1. Verifica se role === 'admin'
2. Se não for admin, retorna 403
3. Se for admin, chama next()

▶ Se é admin → Controllers

📦 ****CONTROLLERS**** - Camada de Controle (recebe requisições HTTP)

▶ ****user.controller.ts**** (UserController)

- register(req, res) → Cria novo usuário

- login(req, res) → Autentica usuário
- getMe(req, res) → Retorna dados do usuário logado
- findAll(req, res) → Lista usuários (admin)
- updateRole(req, res) → Altera role (admin)
- delete(req, res) → Deleta usuário (admin)
- └─▶ Chama: UserService

- ▶ ****item.controller.ts**** (ItemController)
 - findAll(req, res) → Lista itens com filtros
 - findOne(req, res) → Busca item por ID
 - create(req, res) → Cria novo item
 - update(req, res) → Atualiza item
 - delete(req, res) → Deleta item
 - findMyItems(req, res) → Lista itens do usuário
 - └─▶ Chama: ItemService

- ▶ ****proposal.controller.ts**** (ProposalController)
 - findSent(req, res) → Lista propostas enviadas
 - findReceived(req, res) → Lista propostas recebidas
 - create(req, res) → Cria proposta
 - respond(req, res) → Aceita/Recusa proposta
 - └─▶ Chama: ProposalService, NotificationService

- ▶ ****chat.controller.ts**** (ChatController)
 - getConversations(req, res) → Lista conversas
 - getMessages(req, res) → Lista mensagens
 - markAsRead(req, res) → Marca mensagens como lidas
 - getUnreadCount(req, res) → Conta não lidas
 - └─▶ Chama: ChatService

- ▶ ****notification.controller.ts**** (NotificationController)
 - findByUser(req, res) → Lista notificações
 - markAsRead(req, res) → Marca como lida
 - └─▶ Chama: NotificationService

⚙️ ****SERVICES**** - Lógica de Negócio

- ▶ ****user.service.ts**** (UserService)
 - **Métodos:****
 - register(nome, email, senha)
 1. Valida dados
 2. Verifica se email já existe
 3. Hash da senha com bcrypt
 4. Cria usuário no banco

5. Gera token JWT

6. Retorna { user, token }

- login(email, senha)

1. Busca usuário por email

2. Compara senha com bcrypt

3. Gera token JWT

4. Retorna { user, token }

- findAll() → Busca todos usuários

- findById(id) → Busca usuário por ID

- update(id, data) → Atualiza usuário

- delete(id) → Deleta usuário

- updateRole(id, role) → Altera role

▶ Acessa: UserRepository (TypeORM)

▶ ****item.service.ts**** (ItemService)

****Métodos:****

- findAll(filters)

1. Cria query com TypeORM QueryBuilder

2. Aplica filtros (categoria, search, status)

3. Aplica paginação

4. Retorna { data, pagination }

- findById(id) → Busca item + relacionamentos

- create(userId, data, files)

1. Processa upload de imagens

2. Salva URLs das imagens

3. Cria item no banco

- update(id, userId, data) → Atualiza item

- delete(id, userId) → Deleta item

- findByUser(userId) → Lista itens do usuário

▶ Acessa: ItemRepository (TypeORM)

▶ ****proposal.service.ts**** (ProposalService)

****Métodos:****

- create(data)

1. Valida se itens existem

2. Valida se usuário não está propondo pra si mesmo

3. Cria proposta no banco

4. Cria notificação para o dono do item

5. Retorna proposta criada

- findSent(userId) → Lista propostas enviadas
- findReceived(userId) → Lista propostas recebidas
- respond(id, userId, status)
 1. Busca proposta
 2. Verifica se usuário é dono do item
 3. Atualiza status (aceita/recusada)
 4. Cria notificação para proponente
 5. Se aceita, marca item como indisponível

▶ Acessa: ProposalRepository, NotificationService

▶ **chat.service.ts** (ChatService)

****Métodos:****

- createMessage(senderId, receiverId, itemId, conteudo)
 1. Cria mensagem no banco
 2. Normaliza retorno (content + conteudo)
 3. Retorna mensagem com relacionamentos
- getConversations(userId)
 1. Busca mensagens do usuário
 2. Agrupa por conversa (otherUser + item)
 3. Conta não lidas por conversa
 4. Retorna lista de conversas
- getMessages(userId, otherUserId, itemId)
 1. Busca mensagens entre os dois usuários sobre o item
 2. Ordena por data (ASC)
 3. Normaliza retorno
- markAsRead(userId, otherUserId, itemId)
 1. Atualiza campo 'lida' para true
 2. WHERE receiverId = userId AND senderId = otherUserId
- countUnread(userId) → Conta mensagens não lidas

▶ Acessa: ChatMessageRepository

▶ **notification.service.ts** (NotificationService)

****Métodos:****

- create(userId, type, title, message, link, metadata)
 1. Cria notificação no banco

2. TODO: Enviar via WebSocket quando conectado

- findByUser(userId, limit) → Lista notificações
- markAsRead(id, userId) → Marca como lida
- markAllAsRead(userId) → Marca todas como lidas
- getUnreadCount(userId) → Conta não lidas

****Métodos Helper:****

- notifyNewProposal(itemOwnerId, itemTitle, proposerName)
- notifyProposalAccepted(proposerId, itemTitle)
- notifyProposalRejected(proposerId, itemTitle)
- notifyNewMessage(userId, senderName)
- notifyNewRating(userId, raterName, stars)

▶ Acessa: NotificationRepository

■ ****DATABASE**** - TypeORM + SQLite

▶ ****backend/src/config/database.ts****

- Configura DataSource do TypeORM
- Tipo: SQLite
- Arquivo: database.sqlite
- synchronize: true (cria/atualiza tabelas automaticamente)
- Entidades registradas: [User, Item, Proposal, ChatMessage, Notification, Rating, Report]
- Cria admin padrão ao iniciar

▶ ****ENTIDADES**** (backend/src/entities/*.ts)

▶ ****User.ts**** - Entidade Usuário

****Colunas:****

- id: number (PK, auto-increment)
- nome: string
- email: string (unique)
- senha: string (hash bcrypt)
- role: 'common' | 'verified' | 'admin'
- telefone: string (nullable)
- cidade: string (nullable)
- estado: string (nullable)
- latitude: number (nullable)
- longitude: number (nullable)
- createdAt: Date
- updatedAt: Date

****Relacionamentos:****

- items: OneToMany → Item[]
- sentProposals: OneToMany → Proposal[]
- receivedProposals: OneToMany → Proposal[]
- sentMessages: OneToMany → ChatMessage[]
- receivedMessages: OneToMany → ChatMessage[]
- notifications: OneToMany → Notification[]

▶ Método: toJSON() → Remove senha do retorno

▶ ****Item.ts**** - Entidade Item

****Colunas:****

- id: number (PK)
- titulo: string
- descricao: text
- categoria: string
- imagens: json (array de URLs)
- disponivel: boolean (default: true)
- latitude: decimal (nullable)
- longitude: decimal (nullable)
- cidade: string (nullable)
- estado: string (nullable)
- createdAt: Date
- updatedAt: Date

****Relacionamentos:****

- usuario: ManyToOne → User
- proposalsDesejado: OneToMany → Proposal[]
- proposalsOferecido: OneToMany → Proposal[]

▶ Cascade: onDelete CASCADE

▶ ****Proposal.ts**** - Entidade Proposta

****Colunas:****

- id: number (PK)
- status: 'pendente' | 'aceita' | 'recusada'
- mensagem: text (nullable)
- createdAt: Date
- updatedAt: Date

****Relacionamentos:****

- proponente: ManyToOne → User (quem fez a proposta)
- proprietario: ManyToOne → User (dono do item desejado)
- itemDesejado: ManyToOne → Item (item que o proponente quer)
- itemOferecido: ManyToOne → Item (item que o proponente oferece)

▶ Cascade: onDelete CASCADE

▶ ****ChatMessage.ts**** - Entidade Mensagem de Chat

****Colunas:****

- id: number (PK)
- conteudo: text (conteúdo da mensagem)
- lida: boolean (default: false)
- createdAt: Date
- updatedAt: Date

****Relacionamentos:****

- sender: ManyToOne → User (remetente)
- receiver: ManyToOne → User (destinatário)
- item: ManyToOne → Item (item sobre o qual conversam)

▶ Índices: (senderId, receiverId, itemId) para performance

▶ ****Notification.ts**** - Entidade Notificação

****Colunas:****

- id: number (PK)
- type: enum NotificationType
(NEW_PROPOSAL, PROPOSAL_ACCEPTED, PROPOSAL_REJECTED,
NEW_MESSAGE, NEW_RATING)
- title: string
- message: text
- link: string (nullable, URL para redirecionar)
- metadata: json (nullable, dados extras)
- read: boolean (default: false)
- createdAt: Date

****Relacionamentos:****

- user: ManyToOne → User

▶ Cascade: onDelete CASCADE

▶ ****Rating.ts**** - Entidade Avaliação (futuro)

****Colunas:****

- id: number (PK)
- stars: number (1-5)
- comentario: text (nullable)
- createdAt: Date

****Relacionamentos:****

- avaliador: ManyToOne → User

- avaliado: ManyToOne → User

- ▶ Constraint: unique(avaliador, avaliado) - um não pode avaliar o mesmo várias vezes

- ▶ ****Report.ts**** - Entidade Denúncia (futuro)

- **Colunas:****

- id: number (PK)

- tipo: 'user' | 'item' | 'proposal'

- motivo: string

- descricao: text

- status: 'pendente' | 'analizada' | 'resolvida'

- createdAt: Date

- **Relacionamentos:****

- denunciante: ManyToOne → User

- targetId: number (ID do alvo da denúncia)

- ▶ Cascade: onDelete CASCADE



WEBSOCKET (Socket.IO) - Chat em Tempo Real



▶ ****backend/src/websocket/chat.socket.ts**** - ChatSocketHandler

🚩 ****INICIALIZAÇÃO****

- Instância Singleton
- Recebe servidor Socket.IO
- Armazena conexões ativas: Map<userId, Set<socketId>>
- Configura middleware de autenticação
- Configura handlers de eventos

▶ ****MIDDLEWARE DE AUTENTICAÇÃO****

1. Extrai token de handshake.auth.token
2. Verifica JWT
3. Anexa userId ao socket
4. Aceita ou rejeita conexão

▶ ****EVENT HANDLERS****

▶ ****'connection'**** - Novo cliente conectou

1. Registra socket do usuário no Map
2. Entra na room `user:\${userId}`
3. Broadcast: 'user:online' para todos
4. Log: "Usuário X conectado"

▶ ****'user:request_online_list'**** - Cliente pede lista de online

1. Busca todos os userId no Map
2. Emite 'user:online_list' para o cliente

▶ ****'message:send'**** - Cliente envia mensagem

1. Valida dados (receiverId, itemId, content)
2. Chama chatService.createMessage()
3. Normaliza resposta (content + conteudo)
4. Emite 'message:received' para remetente (confirmação)
5. Emite 'message:received' para destinatário (room `user:\${receiverId}`)
6. Atualiza contador: emite 'unread:update' para destinatário

▶ ****'message:read'**** - Cliente marcou mensagens como lidas

1. Chama chatService.markAsRead()
2. Atualiza contador local
3. Emite 'unread:update' para o cliente

▶ ****'user:typing'**** - Cliente está digitando

1. Valida dados (receiverId, itemId, isTyping)
2. Emite 'user:typing' para destinatário
3. Se isTyping=true, auto-remove após 3s

▶ **disconnect** - Cliente desconectou

1. Remove socketId do Map
2. Se não há mais sockets do usuário:
 - Remove userId do Map
 - Broadcast: 'user:offline' para todos
3. Log: "Usuário X desconectado"

▶ **MÉTODOS PÚBLICOS**

- sendNotification(userId, event, data)
 - Emite evento para room `user:\${userId}`
- isUserOnline(userId): boolean
- getOnlineUsersCount(): number
- getOnlineUsers(): number[]

🗄️ **FRONTEND (Socket.IO Client)** - stores/chat.ts

▶ **setupSocketListeners** - Configura listeners

▶ **connect** - Conectado ao servidor

1. Log: "WebSocket conectado"
2. Emite: 'user:request_online_list'

▶ **disconnect** - Desconectado

1. Log: "WebSocket desconectado"

▶ **user:online_list** - Recebe lista de online

1. Atualiza onlineUsers Set
2. Log: "Lista de usuários online"

▶ **message:received** - Nova mensagem recebida

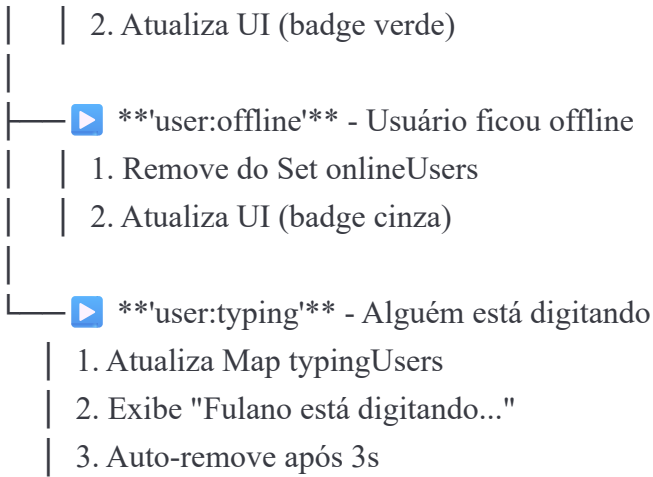
1. Normaliza campos (content/conteudo)
2. Atualiza conversas (fetchConversations)
3. Atualiza notificações
4. Se conversa está aberta:
 - Adiciona à lista de mensagens (evita duplicatas)
 - Marca como lida automaticamente

▶ **unread:update** - Contador de não lidas atualizado

1. Recarrega conversas
2. Atualiza badge

▶ **user:online** - Usuário ficou online

1. Adiciona ao Set onlineUsers

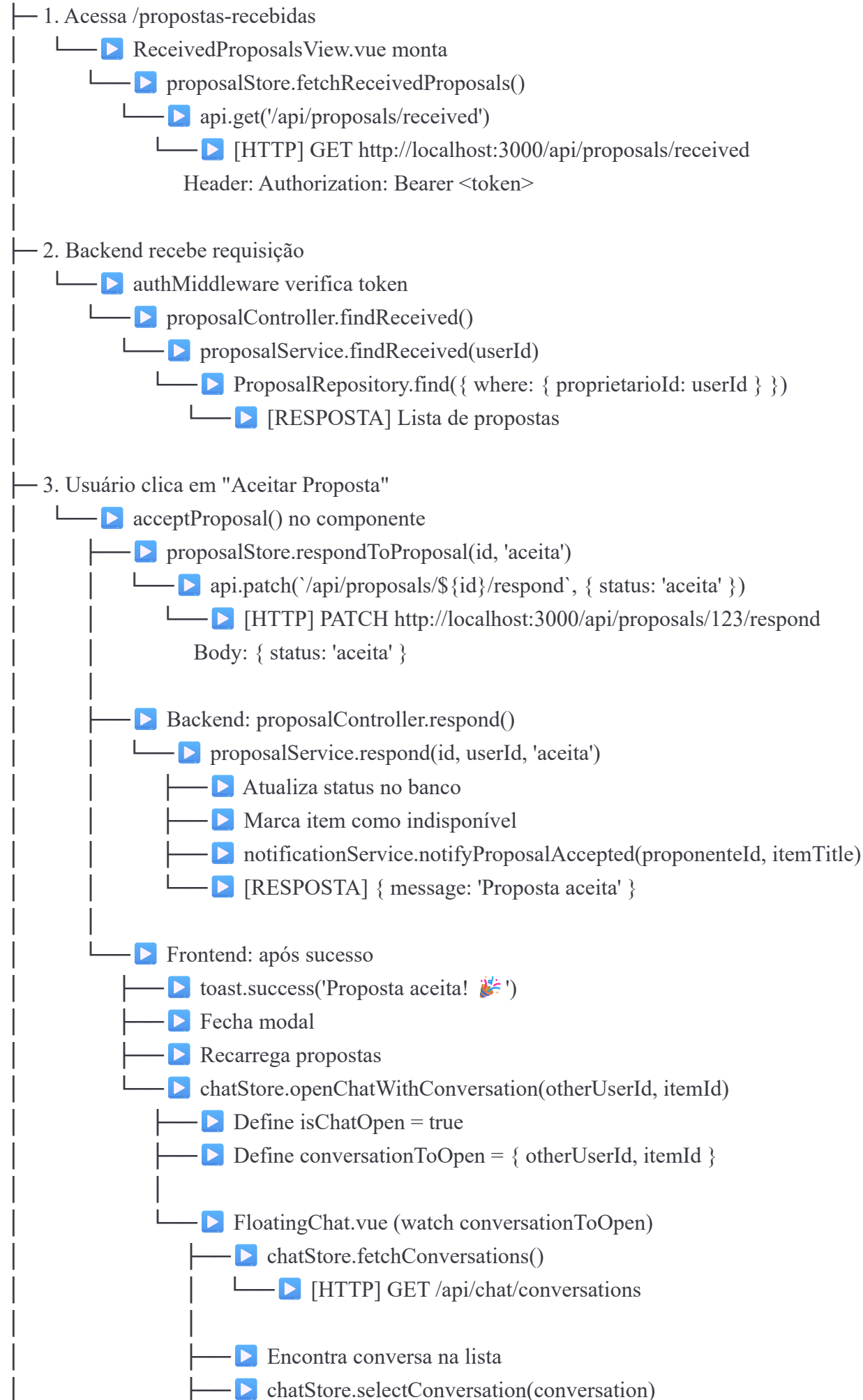


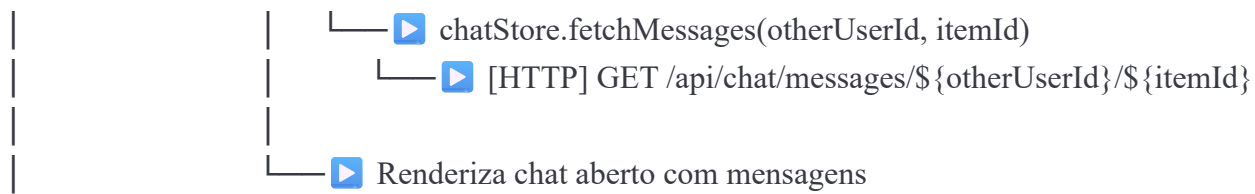
FLUXO COMPLETO DE EXEMPLO

Exemplo 1: Aceitar Proposta e Abrir Chat



USUÁRIO (Frontend)





4. Usuário digita mensagem e envia

